

Search Algorithms Report

Linear Search

Objective: Demonstrate the search algorithm.

Description: Linear Search checks each element one by one until the target is found or the list ends. Best: $O(1)$, Average: $O(n)$, Worst: $O(n)$.

Source Code:

```
import time

def linear_search(arr, target):
    # Traverse the list to find the target
    for i in range(len(arr)):
        if arr[i] == target:
            return i # Return index if found
    return -1 # Return -1 if not found

# --- User Input Section ---
if __name__ == "__main__":
    # Request numbers from the user separated by spaces
    user_input = input("Enter numbers separated by spaces (e.g., 5 2 9 1 7): ")

    try:
        # Convert the string input into a list of integers
        numbers = [int(x) for x in user_input.split()]

        print("\nOriginal List:", numbers)

        # Ask the user for the target number to search
        target = int(input("Enter the number to search: "))

        # Record the time right before starting the search
        start_time = time.perf_counter()

        # Run the linear search algorithm
        result_index = linear_search(numbers, target)

        # Record the time right after the search finishes
        end_time = time.perf_counter()

        # Calculate execution time (End - Start)
        execution_time = end_time - start_time

        # Print Results
        if result_index != -1:
            print(f"Number {target} found at index {result_index}.")
        else:
            print(f"Number {target} not found in the list.")

        print(f"Execution Time: {execution_time:.6f} seconds")

        # --- Time Complexity Info ---
        print("\n--- Time Complexity Analysis ---")
        print("Best Case (Target at First Position):  $O(1)$ ")
        print("Worst Case (Target Not Found or at End):  $O(n)$ ")
        print("Average Case:  $O(n)$ ")

    except ValueError:
        print("Please enter valid integers separated by spaces.")
```

Conclusion: The program successfully performs the search and reports execution time.

Binary Search

Objective: Demonstrate the search algorithm.

Description: Binary Search repeatedly divides a sorted array in half to locate the target. Best: $O(1)$, Average: $O(\log n)$, Worst: $O(\log n)$.

Source Code:

```
import time

def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        # Check if target is at mid
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1 # Target not found

# --- User Input Section ---
if __name__ == "__main__":
    # Request numbers from the user separated by spaces
    user_input = input("Enter numbers separated by spaces (e.g., 5 2 9 1 7): ")

    try:
        # Convert the string input into a list of integers
```

Conclusion: The program successfully performs the search and reports execution time.